

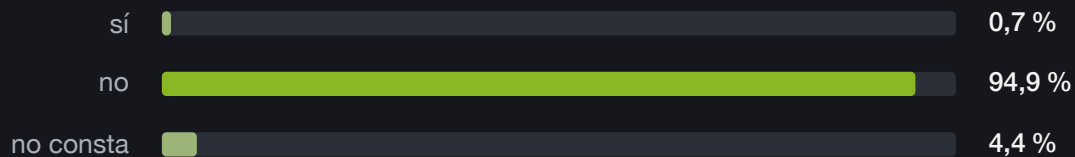
Modelos de decisión y ajuste fino

Una introducción para programadores que ya han usado modelos de lenguaje

RESPUESTA REAL DE KEV-4B

«El interesado presentó la solicitud el 15 de enero, pero no consta el pago de la tasa.»

¿Está la solicitud completa?



QUÉ INCLUYE

- La caja de herramientas escalonada: cuándo usar cada una
- Qué cambia frente a un chat, y la API con peticiones reales
- Cómo funcionan por dentro: logits, softmax, calibración
- El ajuste fino paso a paso, en una máquina propia
- Cinco pruebas en español: BOE, PAWS-X y tres casos de uso
- Veredicto provisional sobre el estado de esta tecnología

Joaquín Herrero Pintado

Creative Codeworks



1. En cinco minutos

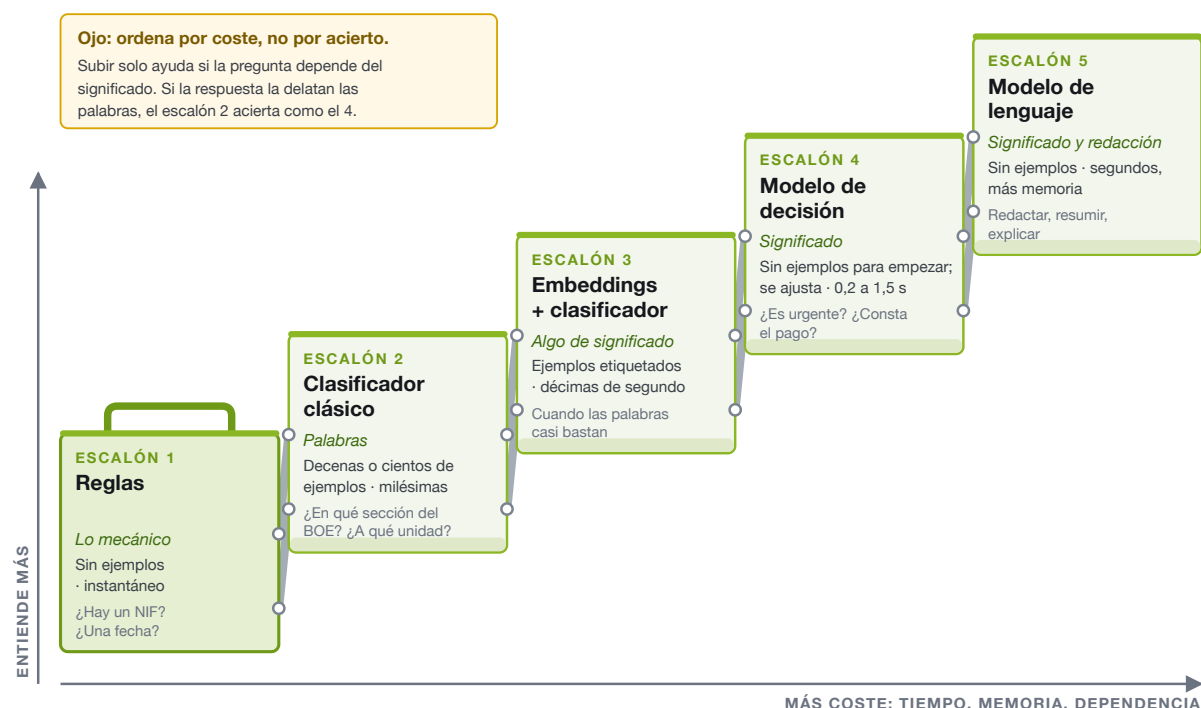
Qué es un modelo de decisión. Un modelo de inteligencia artificial pequeño, de pesos abiertos, que se ejecuta en un ordenador propio y no escribe: recibe un texto, una pregunta cerrada y una lista de respuestas posibles, y devuelve una probabilidad para cada una. «¿Está completa esta solicitud? Sí 1 %, no 95 %, no consta 4 %». No puede inventar una respuesta fuera de la lista y dice cuánta seguridad tiene.

La idea más útil de este informe: la caja de herramientas escalonada. Para responder una pregunta cerrada sobre un texto hay cinco herramientas, de la más barata a la más cara, como los escalones de una caja de herramientas que se abre en bandejas escalonadas. Se empieza por abajo y se sube al escalón siguiente solo cuando el de abajo no llega, midiéndolo.

La caja de herramientas escalonada

Cuándo usar cada herramienta de IA para preguntas cerradas sobre textos. Cada escalón cuesta más y entiende más.

Se empieza por abajo y se sube al escalón siguiente solo si el de abajo no llega, midiéndolo.



© 2026 Joaquín Herrero Pintado · CC BY-SA 4.0 · creativecodeworks.com

Lo que hemos medido. Todo en un Mac mini de 24 GB, sin enviar nada fuera, con cinco pruebas en español:

Pregunta	Depende de	Hasta qué escalón hizo falta subir
¿En qué sección del BOE se publica este texto?	Vocabulario	Escalón 2. El clasificador clásico llega al 95 % con 1.400 ejemplos, igual que un modelo de decisión ajustado durante horas, y con solo 5 ejemplos por sección ya supera a todos los modelos sin ajustar
¿Significan lo mismo estas dos frases?	Significado	Escalón 4. El clásico se queda en el azar incluso con 5.000 ejemplos, y los <i>embeddings</i> , en el 58 %. Un modelo de decisión sin ajustar llega al 78 %, y uno pequeño ajustado con 200 ejemplos pasa del 60 % al 80 %
Triaje del buzón general: unidad, tipo y urgencia	Las dos cosas	Escalón 2 para la unidad, 4 para lo demás. El clásico es el mejor encaminando, pero casi no distingue lo urgente
¿Está completo este expediente?	Significado	Escalón 4. El clásico y los <i>embeddings</i> fallan con los matices («el pago se hará en ventanilla», «el justificante es de otro expediente»)
¿Esta respuesta se apoya en el documento?	Significado	Escalón 4 o 5. Un modelo de decisión pequeño entrenado para esta tarea (Jeff-0.8B) saca un 8,4 sobre 10, y el chat de 9.000 millones, un 9,4; el clásico se queda en un 3,3

Notas de 1 a 10 en los casos de uso (1 es el azar y 10 acertarlo todo; se resta si el modelo falla con mucha seguridad; el detalle está en la sección 5 de la [metodología](#)):

	Triaje de correo	Expedientes	Respuestas apoyadas
Kev-4B (modelo de decisión, 4.000 millones de parámetros)	7,5	8,4	4,9
Jeff-0.8B (modelo de decisión, 800 millones)	3,9	4,4	8,4
Qwen3.5 9B (modelo de chat general, preguntado con letras)	7,7	8,5	9,4
<i>Embeddings</i> y clasificador	6,5	3,0	2,5
Clasificador clásico	5,5	3,3	3,3
Kev-0.8B (modelo de decisión, 800 millones)	5,1	4,7	1,0

Un modelo de chat mediano, bien preguntado, rinde como el mejor modelo de decisión pequeño, pero tarda de tres a cuatro veces más y ocupa más memoria. Ningún modelo de decisión gana en todo: cada uno rinde en lo que se parece a su entrenamiento (Jeff-0.8B es flojo en correo y expedientes y muy bueno comprobando respuestas). Por eso hay que medirlos con la tarea propia.

Veredicto provisional (versión 2.0). Para preguntas que dependen del significado (urgencia, completitud de un expediente, si dos textos dicen lo mismo), los modelos de decisión de 2.000

a 4.000 millones de parámetros **ya son útiles con revisión humana**, en una máquina propia. Para preguntas que se resuelven por el vocabulario, **un clasificador clásico basta** y es mucho más barato. Ninguno está para decidir solo: casi todos se equivocan con demasiada seguridad, así que no se les deben fijar umbrales sin recalibrarlos. Son proyectos de semanas; la madurez sigue siendo su punto débil.

Si solo te llevas tres cosas:

1. Empieza por el escalón más bajo que pueda funcionar.
2. Mide siempre contra un clasificador clásico: si lo iguala, el modelo sobra.
3. No te fíes de la seguridad que declara un modelo hasta haberla comprobado con tus datos.

2. Antes de empezar

Este texto es para quien programa y ya ha usado modelos de lenguaje: has llamado a la API de un chat, has escrito *prompts*, quizá has pedido la respuesta en JSON y has tenido que validarla. No hace falta saber nada de aprendizaje automático.

Al terminar deberías saber:

- qué es un **modelo de decisión**, en qué se parece a un modelo de lenguaje y en qué no;
- cómo se le hace una pregunta y cómo leer lo que devuelve;
- qué significan sus probabilidades y por qué hay que comprobarlas antes de fiarse de ellas;
- qué es el **ajuste fino** (*fine-tuning*), cómo se hace con una máquina propia y qué hay que medir para saber si ha salido bien;
- cuándo conviene un modelo de decisión y cuándo basta algo más sencillo, como un clasificador clásico;
- qué pasó al medirlo de verdad, con textos del BOE, una prueba de significado y tres casos de uso, incluidos los problemas que aparecieron.

Si solo tienes cinco minutos, la sección 1 («En cinco minutos») lo resume. El resto va en cinco partes: **entender** qué son y cómo funcionan; **decidir** cuándo usarlos; **lo que se ha medido**; **hacerlo tú**, con el ajuste fino paso a paso; y un cierre con el veredicto y un glosario. El detalle de cómo se ha medido cada cifra está en una página aparte, enlazada en la sección 12 («Cómo se ha medido»).

El código, los datos de la prueba y un laboratorio para probar varios modelos en tu máquina están en este repositorio:

<https://github.com/joakinen/toolkit-modelos-de-decision>

Funciona como un **toolkit de evaluación de modelos de decisión**: se irá actualizando con los modelos nuevos que salgan. Al final del texto hay un veredicto provisional sobre el estado de esta tecnología. Esta es la versión 2.1, del 29 de septiembre de 2026; la última está siempre en <https://creativecodeworks.com/toolkit-modelos-de-decision.html>, y lo que cambia en cada una, en **CAMBIOS.md**.

3. Qué es un modelo de decisión

Un **modelo de decisión** es un modelo de inteligencia artificial que no escribe. Recibe un texto, una pregunta cerrada y la lista de respuestas posibles, y devuelve **una probabilidad para cada respuesta**. Si lo escribieras como una función, su firma sería esta:

```
def decidir(texto: str, pregunta: str,
            opciones: list[str]) -> dict[str, float]:
    """Una probabilidad por opción. Suman 1.
    Nunca devuelve una opción que no esté en la lista."""
```

Un ejemplo real, con Kev-4B (un modelo de decisión abierto que se ejecuta en un ordenador de sobremesa):

Texto: «El interesado presentó la solicitud el 15 de enero, pero no consta el pago de la tasa.»

Pregunta: ¿Está la solicitud completa?

Respuesta: sí 0,7 % · no 94,9 % · no consta 4,4 %

No redacta una explicación ni puede contestar «depende». Solo reparte la probabilidad entre las opciones que le das.

Admite tres tipos de pregunta. Los nombres de la segunda columna son los que usa la API (vienen de Jev, el primer modelo de este tipo, y Kev los copia para ser compatible):

Tipo	Nombre en la API	Qué devuelve	Ejemplo
Elegir una opción	choice	Probabilidad de cada opción	¿Qué tipo de escrito es: solicitud, alegación, recurso u otro?
Puntuar en una escala	score	Probabilidad de cada nivel	¿Qué urgencia tiene: baja, media o alta?
Sí o no	noyl	Probabilidad de «sí»	¿Menciona el texto un plazo?

A veces se les llama también modelos de «Sistema 1», por la distinción de Daniel Kahneman entre el pensamiento rápido e intuitivo (Sistema 1) y el lento y deliberado (Sistema 2). Un modelo de decisión hace juicios rápidos sobre un texto: no razona en varios pasos ni planifica.

4. Lo que ya sabes de los modelos de lenguaje, y lo que cambia

Un modelo de lenguaje (ChatGPT, Claude, Gemini, Llama, Qwen...) **genera texto**: calcula la probabilidad de cada posible siguiente trozo de palabra (*token*), elige uno, lo añade al texto y repite. Un modelo de decisión parte de uno de esos modelos, pero le quita la parte que escribe (se explica cómo en la sección 6 («Cómo funciona por dentro»)). La tabla resume lo que eso cambia para quien lo usa desde código:

	Modelo de lenguaje	Modelo de decisión
Qué le envías	Un <i>prompt</i> : instrucciones, contenido y formato mezclados en texto libre	Campos fijos: el texto, la pregunta y las opciones
Qué devuelve	Texto, generado <i>token a token</i>	Un JSON con una probabilidad por opción, calculado de una vez
¿Puede salirse de las opciones?	Sí, aunque le pidas JSON: hay que validar la respuesta	No: la salida es siempre una de las opciones
La «temperatura»	Controla la aleatoriedad al elegir el siguiente <i>token</i>	No hay nada que elegir al azar; la temperatura corrige lo seguro que se muestra (sección 7 («Probabilidad, calibración y umbrales»))
Coste	Pagas sobre todo los <i>tokens</i> que genera	Casi no genera nada: el coste es leer la entrada
Tiempo por consulta	Segundos	De décimas de segundo a un segundo, en un ordenador de sobremesa
Cómo se evalúa	Hay que valorar textos, a mano o con otro modelo	Se cuenta cuántas veces acierta

La consecuencia práctica: para **clasificar, encaminar o comprobar**, un modelo de decisión es más barato, más rápido y más fácil de controlar que un chat. Para **redactar, resumir o explicar**, no sirve.

4.1. ¿No basta con pedirle a un chat que responda con una letra?

Es lo primero que se le ocurre a cualquiera que ha usado la API de un chat: escribir las opciones como A, B, C, pedir «responde solo con la letra» y leer la probabilidad que el modelo da a cada letra en el primer *token* (muchas APIs la devuelven con la opción `logprobs`). Funciona, y el laboratorio del repositorio lo usa para comparar un modelo de chat general de 9.000 millones de parámetros (Qwen3.5 9B) con los modelos de decisión.

Tiene tres problemas. El modelo no se entrenó para esto, así que su probabilidad sobre las letras no está pensada para significar «cuánto acierto»; depende de detalles del *prompt*, como el orden de las opciones; y sigue pagando el coste de un modelo grande. En la prueba con textos del BOE de la sección 13 («Una pregunta de vocabulario: el BOE»), ese modelo de 9.000 millones

acierta el 66 %; un modelo de decisión de 800 millones, ajustado con ejemplos, acierta el 95 %. En preguntas que dependen del significado, en cambio, ese mismo chat rinde como el mejor modelo de decisión pequeño, aunque más despacio (se cuenta en la sección 15 («Tres casos de uso»)).

5. Cómo se llama: la API

Kev, el modelo abierto que se usa en este texto, se sirve con un servidor HTTP local que imita la API de Jev. La petición es un JSON con el texto (state) y un diccionario de preguntas; cada pregunta lleva un identificador que eliges tú:

```
{
  "state": "El interesado presentó la solicitud el 15 de enero,
           pero no consta el pago de la tasa.",
  "questions": {
    "completa": {"type": "choice",
                 "instructions": "¿Está la solicitud completa?",
                 "criteria": {"sí": null, "no": null,
                             "no consta": "El texto no permite saberlo"}},
    "plazo": {"type": "noul",
              "instructions": "¿Menciona el texto un plazo?"},
    "urgencia": {"type": "score",
                 "instructions": "¿Qué urgencia tiene?",
                 "criteria": ["baja", "media", "alta"]}
  }
}
```

En criteria van las opciones. En choice, cada opción puede llevar una descripción corta (aquí, «no consta») o null. En score, las opciones son los niveles de la escala, de menor a mayor. En noul no hacen falta.

Desde Python es una llamada normal:

```
import requests

peticion = {...} # el JSON de arriba
url = "http://127.0.0.1:8009/v1/systemone" # Kev-4B servido en local
r = requests.post(url, json=peticion, timeout=60)
print(r.json()["answers"]["completa"]["probabilities"])
# {'sí': 0.0072, 'no': 0.9488, 'no consta': 0.044}
```

Esta es la respuesta completa que dio Kev-4B, recortada a lo esencial:

```
{
  "answers": {
    "completa": {"type": "choice", "choice": "no",
      "probabilities": {"sí": 0.0072, "no": 0.9488,
        "no consta": 0.044}},
    "plazo": {"type": "noul", "noul": 0.729},
    "urgencia": {"type": "score", "score": 0.7953,
      "probabilities": {"0": 0.4677, "1": 0.2692, "2": 0.2631}}
  }
}
```

Cómo leerla:

- **completa:** elige «no» con un 94,9 %. Es la respuesta correcta y la da con mucha seguridad.
- **plazo:** devuelve la probabilidad de «sí», un 72,9 %. El texto trae una fecha, pero no un plazo, así que la respuesta correcta sería «no». El modelo se equivoca, pero con una seguridad moderada. Esa duda es información: una respuesta al 73 % no debería aceptarse sin que la mire una persona.
- **urgencia:** en una escala, devuelve la probabilidad de cada nivel (numerados desde 0) y su media (0,80, entre «baja» y «media»). El reparto (47 %, 27 %, 26 %) dice que no lo tiene claro, lo que tiene sentido: el texto no da pistas sobre la urgencia.

La misma petición a Kev-0.8B, el modelo pequeño, da en completa un 48 % a «no consta», un 29 % a «no» y un 23 % a «sí». Acierta menos y duda más. Es un patrón que se repite: los modelos más pequeños son más baratos y rápidos, pero dudan más.

Tres detalles prácticos:

- **Varias preguntas en una llamada.** El modelo lee el texto una vez y responde cada pregunta por separado, sin que la respuesta a una influya en las demás.
- **Las opciones son el nuevo *prompt*.** Escribirlas bien es lo que más influye en el resultado: que no se solapen, que cubran todos los casos y que haya un «no consta» cuando el texto puede no decir nada.
- **La respuesta trae también un campo *confidence*.** No es la probabilidad de acertar: es una transformación de la probabilidad más alta. Para decidir, usa las probabilidades.

6. Cómo funciona por dentro

No hace falta entender esto para usarlo, pero ayuda a entender sus límites y qué cambia al ajustarlo.

1. Se parte de un modelo de lenguaje ya entrenado. Kev-0.8B y Kev-4B parten de los modelos base de Qwen3.5. Un modelo de lenguaje ha leído enormes cantidades de texto y ya «entiende» el idioma: sabe qué es una negación, un plazo o una fecha. Esa comprensión es lo que se aprovecha.

2. Se le cambia la salida. El texto, la pregunta y las opciones se escriben en una sola secuencia con marcas:

```

<state> ...el texto...
<q> ¿Está la solicitud completa?
<opt> sí </opt> <opt> no </opt> <opt> no consta </opt> <decide>

```

El modelo de lenguaje la lee entera, como leería un *prompt*, pero no se le pide que escriba nada. En lugar de la pieza que calcula el siguiente *token*, se le añade una pieza pequeña, la **cabeza**, que compara lo que el modelo «tiene en mente» al llegar a <decide> con lo que tenía al terminar cada opción, y da **una puntuación por opción**. A esas puntuaciones se les llama **logits**. Después, una función llamada **softmax** las convierte en probabilidades que suman 1. En pseudocódigo:

```

# un número por opción
logits = [cabeza(estado_en_decide, estado_al_final_de(op))
          for op in opciones]
probabilidades = softmax(logits)

def softmax(z):
    e = [math.exp(x) for x in z]
    return [x / sum(e) for x in e]

```

Con tres opciones y *logits* 2,0, 0,5 y -1,0, softmax da 78,6 %, 17,5 % y 3,9 %. Cuanto mayor es la diferencia entre los *logits*, más seguro se muestra el modelo.

3. Se entrena con muchísimos ejemplos resueltos. Cada ejemplo es un texto, una pregunta, unas opciones y la respuesta correcta. Salen de colecciones públicas (noticias, reseñas, consultas de clientes, preguntas sobre un texto, inferencia) y de ejemplos generados con reglas, en los que la respuesta correcta se conoce por construcción (por ejemplo, si una fecha cae dentro de un plazo). El entrenamiento compara la probabilidad que el modelo dio a la respuesta correcta con la que debería haber dado, y ajusta los números del modelo para acercarlas. El error de cada ejemplo se mide con la **log-loss**: menos el logaritmo de la probabilidad que dio a la respuesta correcta.

Probabilidad dada a la respuesta correcta	Log-loss
99 %	0,01
90 %	0,11
50 %	0,69
10 %	2,30
1 %	4,61

Fíjate en la forma: dudar cuesta poco, pero **equivocarse con seguridad cuesta mucho**. Dar un 1 % a la respuesta correcta cuesta 40 veces más que darle un 90 %. Por eso el modelo aprende a repartir bien la probabilidad y no solo a acertar. La log-loss media sobre un conjunto de casos es también una de las medidas que se usan para comparar modelos.

Durante el entrenamiento se toman precauciones para que aprenda la tarea y no atajos:

- **Se baraja el orden de las opciones**, para que no aprenda que «la buena suele ser la primera».
- **Se incluyen casos en los que la respuesta no está en el texto**, para que aprenda a decir «no consta» en lugar de adivinar.

- **Se aparta una parte de los ejemplos que el modelo nunca ve al entrenar**, para medir con ellos cuánto acierta de verdad y para corregir al final lo seguro que se muestra (sección 7 («Probabilidad, calibración y umbrales»)).

Todo esto lo hace quien publica el modelo. Lo que puedes hacer tú es el paso siguiente: ajustarlo con tus propios casos.

7. Probabilidad, calibración y umbrales

Lo más útil de estos modelos no es la respuesta, sino **cuánta seguridad tiene en ella**. Con una probabilidad fiable puedes repartir el trabajo con una regla sencilla:

```
p = max(respuesta["probabilities"].values())
if p >= 0.95:
    proponer_automaticamente(respuesta)  # la persona solo confirma
else:
    enviar_a_revision(respuesta)         # la persona decide
```

Para que esa regla funcione, la probabilidad tiene que significar lo que dice: de todas las veces que el modelo dice «95 %», debe acertar unas 95 de cada 100. Eso se llama **calibración**. Un modelo que acierta mucho pero falla a menudo con un 99 % de seguridad es peor para este uso que uno que acierta menos pero sabe cuándo duda, porque sus errores se cuelan por la puerta automática.

Cómo se mide. Se usan dos cifras en este texto:

- **Fallos con seguridad alta:** de las respuestas que el modelo da con un 90 % o más, cuántas son erróneas. Es la que más importa si vas a poner un umbral.
- **ECE** (*expected calibration error*, error de calibración esperado): se agrupan las respuestas por tramos de seguridad (del 90 al 100 %, del 80 al 90 %...) y se compara, en cada tramo, la seguridad media con el acierto real. La ECE es la media de esas diferencias. Cero es la calibración perfecta.

Cómo se corrige: la temperatura. Si un modelo se muestra demasiado seguro, se divide cada *logit* por un número T mayor que 1 antes de aplicar softmax:

```
probabilidades = softmax([z / T for z in logits])
```

Con los *logits* del ejemplo anterior (2,0, 0,5 y -1,0), con $T = 2$ las probabilidades pasan de 78,6 %, 17,5 % y 3,9 % a 59,0 %, 27,9 % y 13,2 %. La opción ganadora sigue siendo la misma, así que **el acierto no cambia**: solo cambia lo seguro que se muestra. T se elige con un conjunto de casos apartado, buscando la que da la menor log-loss. Kev trae ya la suya (2,41 en el 4B, 2,35 en el 0.8B).

Ojo con el nombre: en un modelo de lenguaje, la «temperatura» controla la aleatoriedad al generar. Aquí no se genera nada; es la misma operación matemática, pero sirve para corregir la seguridad, no para dar variedad.

El umbral se elige con datos, no a ojo. Con casos ya resueltos se mira qué porcentaje de error hay por encima de cada umbral posible y cuántos casos quedarían para revisión, y se elige el

equilibrio que convenga. Esos casos tienen que ser distintos de los que se usaron para entrenar y para calibrar: si no, el umbral se ajusta a ellos y promete menos error del que habrá.

PARTE III · DECIDIR

8. La caja de herramientas escalonada

Un modelo de decisión no es la única forma de responder una pregunta cerrada sobre un texto. Hay al menos cinco, y las pruebas de este informe dicen que ninguna gana siempre. Ordenadas de la más barata a la más cara:

Escalón	Qué es	Qué necesita	Qué entiende	Coste por texto
1. Reglas	Expresiones regulares y condiciones escritas a mano	Escribir la regla	Lo mecánico: hay un NIF, hay una fecha	Nada
2. Clasificador clásico	TF-IDF y regresión logística: aprende qué palabras van con cada respuesta	Decenas o cientos de ejemplos etiquetados	Palabras, no significado	Milésimas de segundo, sin GPU
3. <i>Embeddings</i> y clasificador	Un modelo pequeño convierte el texto en un vector y un clasificador aprende encima	Ejemplos etiquetados	Algo de significado	Décimas de segundo
4. Modelo de decisión	Un modelo de lenguaje pequeño reentrenado para elegir entre opciones	Nada para empezar; cientos de ejemplos si se ajusta	Significado	De 0,2 a 1,5 segundos
5. Modelo de lenguaje	Un chat general, preguntado para que conteste con una letra	Nada	Significado, y además sabe redactar	Varios segundos y más memoria

La regla: empieza por abajo y sube al escalón siguiente solo si el de abajo no llega, mi-diéndolo. Cada escalón cuesta más en tiempo, en memoria y en dependencia de un modelo concreto. Subir solo compensa si la pregunta depende del significado.

Cómo elegir, en cuatro preguntas:

1. ¿Es mecánico? Reglas.
2. ¿La respuesta la delatan las palabras y tienes ejemplos etiquetados? Clasificador clásico. Si no llega, prueba con *embeddings* antes de subir otro escalón.

3. ¿Depende del sentido (negaciones, plazos, algo citado pero no aportado), no tienes ejemplos o la pregunta cambia a menudo? Modelo de decisión. Si tienes unos cientos de ejemplos, ajústalo.
4. ¿Hay que redactar, resumir o explicar? Modelo de lenguaje.

Ojo: la caja ordena por coste, no por acierto. En la prueba del BOE, un clasificador clásico entrenado en nueve segundos acierta tanto como un modelo de decisión ajustado durante horas. En el triaje de correo, el clásico es el que mejor encamina a la unidad correcta. Subir de escalón en una pregunta de vocabulario no mejora nada: solo encarece.

Los escalones se combinan. En un buzón general, lo razonable sería que el clasificador clásico decidiera la unidad (rápido y fiable, porque el tema lo delatan las palabras), que un modelo de decisión decidiera la urgencia y el tipo de correo (que dependen del sentido) y que un modelo de lenguaje redactara, si hace falta, el borrador de la respuesta.

Las pruebas de las secciones 13 a 15 muestran hasta qué escalón hizo falta subir en cada caso. La figura de la sección 1 («En cinco minutos») la resume; se puede reutilizar citando la fuente (CC BY-SA 4.0).

9. Qué tareas hacen bien

Todo lo que se pueda formular como una pregunta cerrada sobre un texto:

- **Clasificar:** qué tipo de documento es, a qué materia pertenece.
- **Encaminar:** a qué unidad, cola o persona debe ir.
- **Comprobar:** si falta un documento, si se cumple una condición, si se menciona un requisito.
- **Detectar:** urgencia, reclamación, queja, plazo, datos personales.
- **Priorizar:** puntuar para ordenar una cola de trabajo.
- **Verificar a otro sistema:** comprobar si la respuesta que ha dado otro programa (o un chat) es correcta o está sustentada en el texto.
- **Abstenerse:** responder «no consta» cuando el texto no dice nada, en lugar de suponer.

10. Posibles usos en una administración

Algunos ejemplos, escritos como la pregunta que se le haría al modelo:

Uso	Pregunta	Opciones
Registro de entrada	¿Qué tipo de escrito es?	solicitud · alegación · recurso · consulta · otro
Reparto	¿Qué unidad debe tramitarlo?	la lista de unidades
Compleitud	¿Aporta la documentación obligatoria?	sí · no · no consta
Plazos	¿Se presentó dentro de plazo según el texto?	sí · no · no consta
Urgencia	¿Con qué urgencia hay que atenderlo?	escala del 1 al 5

Uso	Pregunta	Opciones
Consultas	¿La respuesta a esta consulta está en la documentación publicada?	sí · no
Calidad	¿Cita la resolución la norma aplicable?	sí · no
Revisión	¿Contiene datos personales que haya que anonimizar?	sí · no · no consta

No todos piden el mismo escalón de la caja escalonada: el tipo de escrito o la unidad suelen delatarlos las palabras, y ahí puede bastar un clasificador clásico; la completitud, los plazos o la urgencia dependen del sentido, y ahí es donde un modelo de decisión aporta. Tres de estos usos se han medido: se cuentan en la sección 15 («Tres casos de uso»).

En todos ellos el modelo no sustituye a quien tramita: **ordena, filtra y señala**, y deja la decisión a una persona cuando hay duda o cuando la decisión tiene efectos sobre alguien.

También encajan bien **delante de un modelo de lenguaje**: el modelo de decisión, barato y rápido, clasifica todas las entradas, y solo las que lo necesitan pasan a un chat, más caro, para redactar un borrador.

11. Qué no hacen, y con qué cuidado usarlos

- **No redactan, no resumen, no explican.** Solo eligen entre opciones.
- **A veces sobran.** Si la respuesta la delatan las palabras y hay ejemplos, un clasificador clásico acierta lo mismo por una fracción del coste. Mídelo siempre como referencia.
- **Dependen de cómo se escriban las opciones.** Opciones ambiguas o que se solapan dan resultados pobres.
- **Fallan con la ironía, los dobles sentidos y los razonamientos largos** (contar días hábiles, encadenar varias condiciones). Ahí conviene que el modelo dude, y medir si lo hace.
- **Hay que medirlos con casos propios.** Las cifras de quien publica un modelo se obtienen con sus datos, no con los tuyos.
- **Ajustarlos tiene coste y riesgos.** Con pocos ejemplos aprenden poco, y el ajuste puede estropear la calibración aunque el acierto no baje.
- **Las decisiones con efectos sobre personas requieren intervención humana.** El Reglamento General de Protección de Datos (artículo 22) limita las decisiones basadas únicamente en tratamiento automatizado, y la Ley 40/2015 (artículo 41) exige que la actuación administrativa automatizada tenga un órgano responsable definido. Usados como apoyo a la tramitación, estos límites se respetan con facilidad; usados para resolver, no.

12. Cómo se ha medido

Esta parte cuenta los resultados. El detalle de cómo se obtuvo cada cifra (de dónde salen los datos, cómo se reparten, qué se comprueba y qué límites tiene cada prueba) está en la página de metodología:

<https://joakinen.github.io/toolkit-modelos-de-decision/metodologia.html>

Lo esencial cabe en cinco ideas:

- **El diseño se fija antes de medir.** Qué se pregunta, con qué datos y cómo se puntúa se escribe antes de pasar ningún modelo. Lo que se cambia después se anota con la fecha y el motivo.
- **Entrenar, calibrar y medir usan casos distintos.** Ningún modelo se mide con casos que haya visto al prepararse. Con textos fechados, se entrena con lo antiguo y se mide con lo reciente.
- **Siempre hay dos referencias:** un clasificador clásico y *embeddings* con clasificador, entrenados con los mismos ejemplos que los modelos ajustados.
- **Se mide el acierto medio por respuesta posible**, no el acierto a secas: un modelo que dijera siempre la respuesta más frecuente no puede sacar buena nota.
- **Las diferencias llevan intervalo de confianza del 95 %** (*bootstrap*, 2.000 sorteos). Si el intervalo no incluye el cero, la diferencia no se explica por azar.

Todo se ejecutó en un Mac mini con chip M4 Pro y 24 GB de memoria, con modelos de pesos abiertos y sin enviar nada fuera de la máquina.

13. Una pregunta de vocabulario: el BOE

Para comprobar si todo esto funciona con textos administrativos reales, se ha seguido el proceso del ajuste fino con datos públicos del Boletín Oficial del Estado. Es una clasificación documental muy parecida a la de un registro de entrada: leer un texto y decir de qué tipo es.

La pregunta. *¿En qué apartado del BOE se publica este texto?* Las opciones son las siete secciones del sumario:

Apartado	Qué contiene
I	Disposiciones generales (leyes, reales decretos, órdenes de alcance general)
II.A	Nombramientos, situaciones e incidencias del personal
II.B	Oposiciones y concursos
III	Otras disposiciones (actos concretos: convenios, subvenciones, resoluciones)
IV	Administración de Justicia
V.A	Anuncios de contratación del sector público
V.B	Otros anuncios oficiales

Los datos. Se descargaron de la API de datos abiertos del BOE. La respuesta correcta es la sección en que publicó el texto el propio BOE, así que es fiable por construcción. Al modelo se le da solo el cuerpo del texto, sin el título, que a menudo delata la sección. Se entrena con 1.400 textos de enero a junio de 2026 (200 por apartado) y se mide con 350 de julio y agosto (50 por apartado).

Resultados en los 350 textos de prueba (acierto medio por apartado):

Herramienta	Escalón	Acierto	Log-loss
Kev-4B, ajustado con los 1.400 textos	4	96 %	0,22
Clasificador clásico, con los mismos 1.400 textos	2	95 %	0,15
Kev-0.8B, ajustado con los 1.400 textos	4	95 %	0,35
<i>Embeddings</i> y clasificador, con los mismos textos	3	93 %	0,22
Réplica abierta de Jev de 2.000 millones, sin ajustar	4	73 %	1,24
Kev-4B, sin ajustar	4	69 %	0,83
Jeff-0.8B, sin ajustar	4	67 %	0,93
Qwen3.5 9B, modelo de chat general, con letras	5	66 %	0,78
Réplica abierta de Jev de 800 millones, sin ajustar	4	64 %	1,10
Kev-0.8B, sin ajustar	4	49 %	1,32
Jeff-2B, sin ajustar	4	45 %	1,10

«Acierto medio por apartado» es la media de los siete aciertos por apartado: así nadie saca buena nota acertando solo los apartados fáciles.

Lo que enseña:

- **El clasificador clásico iguala a los modelos ajustados.** La diferencia con Kev-4B ajustado (+1,4 puntos a favor de Kev) tiene un intervalo de confianza de $-0,7$ a $+3,6$: incluye el cero, así que no se distingue del azar. Con Kev-0.8B ajustado, la diferencia es prácticamente nula. Y el clásico está mejor calibrado: de las 289 respuestas que da con un 90 % de seguridad o más, falla 2; Kev-4B ajustado falla 8 de 340.
- **Con datos suficientes, el ajuste funciona.** Kev-0.8B pasa del 49 % al 95 % (+45 puntos; intervalo de +41 a +49) y Kev-4B, del 69 % al 96 % (+27; de +24 a +31).
- **Sin ajustar, ningún modelo conoce las convenciones del BOE.** Casi ninguno reconoce las disposiciones generales: las confunden con «otras disposiciones». Distinguir una norma de alcance general de un acto concreto no se deduce del texto; se aprende con ejemplos.
- **No es memoria.** Revisando a mano una muestra de aciertos y separando los textos cuyo tipo de título aparece en el entrenamiento de los que no, el 4B ajustado acierta casi igual en los dos grupos (96,6 % y 95,9 %).

Por qué gana el clásico aquí. Las secciones del BOE las delatan las palabras: «edicto», «juzgado», «licitación», «nombramiento», «convocatoria». Una pregunta así es de vocabulario, y para eso basta el segundo escalón. Cuántos ejemplos le hacen falta lo dice su curva de aprendizaje (10 sorteos por tamaño):

Ejemplos por apartado	Textos en total	Clasificador clásico	<i>Embeddings</i> y clasificador
2	14	66 %	68 %
5	35	75 %	78 %
10	70	83 %	84 %
50	350	92 %	90 %
200	1.400	95 %	93 %

Con solo 5 ejemplos por apartado, 35 textos que se etiquetan en un rato, el clasificador clásico ya supera a todos los modelos de decisión sin ajustar.

Lo que cuesta cada cosa (en el Mac mini, con los mismos 1.400 textos):

	Clasificador clásico	Kev-0.8B ajustado	Kev-4B ajustado
Tiempo de entrenamiento	9 segundos	3 h 29 min	11 h 35 min
Qué se entrena	Una regresión logística	En torno al 1 % de los parámetros (LoRA)	Lo mismo, con la base en media precisión
Memoria máxima	Despreciable, sin GPU	3,3 GB de GPU	9,2 GB de GPU
Tiempo por texto al usarlo	Menos de un milisegundo	Décimas de segundo	En torno a un segundo

A eso hay que sumar reunir los datos: descargar los 1.400 textos del BOE tardó unos 27 minutos. Con el 4B, un equipo de 24 GB está en su límite: llegó a usar unos 14 GB de intercambio a disco.

Fuente de los datos: Agencia Estatal Boletín Oficial del Estado (boe.es), reutilizados según sus condiciones de datos abiertos.

14. Una pregunta de significado: PAWS-X

En el BOE bastaban las palabras. Para ver qué pasa cuando no bastan, hace falta una prueba diseñada contra ellas. **PAWS-X** es una colección pública de pares de oraciones con casi las mismas palabras que unas veces significan lo mismo y otras no. «El vuelo de Madrid a Lima» y «El vuelo de Lima a Madrid» tienen las mismas palabras y no dicen lo mismo. Se hizo precisamente para que fallen los métodos que miran qué palabras hay.

La pregunta. *¿Significa lo mismo que esta otra oración? Sí o no.*

Los datos. La versión en español. Se mide con 400 pares de su partición de prueba, traducida por personas. Para entrenar el clasificador clásico, los *embeddings* y el ajuste se usa una bolsa de 5.000 pares de su partición de entrenamiento.

Resultados en los 400 pares (acierto medio por respuesta; el azar es el 50 %):

Herramienta	Escalón	Ejemplos de la tarea	Acierto
Clasificador clásico	2	50 a 5.000	50 % a 52 %
<i>Embeddings</i> y clasificador	3	50 a 5.000	56 % a 58 %
Kev-0.8B, sin ajustar	4	ninguno	60 %
Réplica de Jev de 800 millones, sin ajustar	4	ninguno	60 %
Réplica de Jev de 2.000 millones, sin ajustar	4	ninguno	68 %
Kev-4B, sin ajustar	4	ninguno	78 %
Kev-0.8B, ajustado con 200 pares	4	200	80 %
Jeff-0.8B, sin ajustar	4	ver nota	80 %
Jeff-2B, sin ajustar	4	ver nota	84 %
Qwen3.5 9B, modelo de chat general, con letras	5	ninguno	79 %

Lo que enseña:

- **El clasificador clásico no aprende la tarea, ni con 5.000 ejemplos.** No es que le falten datos: la información que necesita (el orden, quién hace qué a quién) no está en qué palabras aparecen. Los *embeddings* captan algo, pero poco.
- **Los modelos de decisión sí la resuelven, aunque no todos.** Kev-4B, que nunca vio esta tarea al entrenarse, llega al 78 % sin ajustar, igual que el chat de 9.000 millones (79 %). Los modelos de 800 millones y la réplica de Jev de 2.000 se quedan entre el 60 % y el 68 %, y fallan entre el 30 % y el 43 % de las respuestas que dan con un 90 % o más.
- **Un ajuste pequeño rinde mucho.** Kev-0.8B pasa del 60 % al 80 % con solo 200 pares (+20 puntos; intervalo de +15 a +24), en unos 20 minutos de ajuste. Pero sale demasiado seguro: falla el 19 % de las respuestas que da con un 90 % o más. Otra vez, el ajuste hace que el modelo no sepa cuándo dudar.

Nota sobre Jeff. Jeff se entrenó con 12.000 pares de PAWS en inglés, y los pares de PAWS-X son traducciones de pares de PAWS. Su resultado no es el de un modelo que ve la tarea por primera vez: es la tarea aprendida en inglés y trasladada al español. Kev, en cambio, no vio PAWS al entrenarse.

Fuente de los datos: PAWS-X (Google Research), versión en español.

15. Tres casos de uso

El BOE y PAWS-X miden dos extremos: una pregunta de vocabulario y una de significado. Los usos reales en una administración suelen quedar en medio. Para acercarse a ellos se han preparado tres casos, con las preguntas que se le harían al modelo en el trabajo diario. El diseño de los tres se fijó por escrito antes de medir.

La nota de 1 a 10. Para comparar de un vistazo, cada herramienta recibe una nota por pregunta y la nota del caso es la media. Un 1 es acertar como el azar y un 10, acertarlo todo; si más del 5 % de las respuestas dadas con un 90 % de seguridad o más son errores, se resta un punto, y si son más del 10 %, dos. Las reglas completas y un ejemplo resuelto están en la sección 5 de la metodología.

Los datos son sintéticos. Los correos y los expedientes se escribieron para la prueba, con casi la mitad de casos difíciles a propósito. Los de entrenamiento (para las dos referencias) y los de prueba los escribieron autores distintos, para que no compartan frases hechas. Los datos reales son más variados; por eso el triaje de correo tiene también una prueba privada con correos reales anonimizados, cuyos resultados se publicarán en una versión posterior.

15.1. Triage del buzón general

Correos que llegan al buzón general de un ayuntamiento ficticio, con tres preguntas: **a qué unidad** corresponde (ocho opciones, entre ellas «ninguna»: no es competencia municipal o es publicidad), **qué es** (consulta, trámite, queja, respuesta a un requerimiento o publicidad) y **si es urgente** (un plazo de menos de una semana, un riesgo para personas o un servicio caído). 120 correos de prueba; 160 para entrenar las referencias.

Herramienta	Escalón	Nota	Unidad	Tipo	Urgente
Qwen3.5 9B, chat general con letras	5	7,7	7,3	7,5	8,3
Kev-4B	4	7,5	7,6	7,5	7,4
<i>Embeddings</i> y clasificador	3	6,5	8,6	6,4	4,6
Jeff-2B	4	6,1	6,2	7,3	4,8
Clasificador clásico	2	5,5	8,6	5,7	2,2
Réplica de Jev de 800 millones	4	5,1	4,1	6,2	5,1
Kev-0.8B	4	5,1	6,9	3,0	5,4
Réplica de Jev de 2.000 millones	4	4,6	4,7	4,7	4,4
Jeff-0.8B	4	3,9	4,4	2,0	5,2

Lo que enseña. La pregunta de la unidad es de vocabulario, y ahí ganan los escalones 2 y 3: el tema de un correo lo delatan sus palabras. La urgencia es de significado (hay que calcular un plazo con la fecha del correo, o ver un riesgo), y ahí el clasificador clásico está casi al azar (2,2) mientras Kev-4B y el chat superan el 7. Es el caso que mejor ilustra la combinación de escalones: clásico para encaminar, modelo de decisión para la urgencia y el tipo.

15.2. Completitud de un expediente

Resúmenes de expedientes (licencias, ayudas, certificados) tal como los ve quien los revisa, con cuatro preguntas: **si consta el pago** de la tasa, **si está firmada** la solicitud, **si se aporta el documento de identidad** del interesado (las tres con «sí», «no» o «no consta») y **quién presenta** la solicitud (el interesado, un representante con autorización o uno sin ella). Con trampas: «el pago se hará en ventanilla», «el justificante es de otro expediente», «la firma está pendiente», un DNI citado pero no adjunto. 60 expedientes de prueba; 40 para entrenar las referencias.

Herramienta	Escalón	Nota	Pago	Firma	Identidad	Quién presenta
Qwen3.5 9B, chat general con letras	5	8,5	6,6	8,9	8,5	10
Kev-4B	4	8,4	8,1	8,0	7,4	10
Réplica de Jev de 2.000 millones	4	7,5	7,7	6,2	6,4	9,7
Jeff-2B	4	6,8	5,3	7,0	6,2	8,9
Kev-0.8B	4	4,7	3,2	3,4	2,2	10
Jeff-0.8B	4	4,4	3,3	2,9	2,3	9,0
Réplica de Jev de 800 millones	4	4,2	4,9	1,1	1,4	9,2
Clasificador clásico	2	3,3	4,5	1,9	1,6	5,1
<i>Embeddings</i> y clasificador	3	3,0	4,4	3,2	2,2	2,1

Lo que enseña. Es el caso en que más claramente hace falta el escalón 4. Las referencias fallan porque la respuesta depende de matices que las palabras no recogen: «pago» aparece igual en «pago acreditado» que en «el pago se hará en ventanilla». Los modelos de decisión de 2.000 a 4.000 millones de parámetros llegan a notas de 7 a 8,4; los de 800 millones se quedan cortos, salvo en la pregunta más sencilla (quién presenta).

Un error de diseño, corregido. La primera redacción de la pregunta de identidad decía «del solicitante», que es ambiguo cuando presenta un representante: quien preparó los datos entendió el titular, y los modelos, quien presenta. Se detectó al revisar los casos en que los dos mejores modelos coincidían contra la respuesta esperada. Se cambió la redacción («del interesado, el titular del trámite, no su representante») y se volvió a medir todo. Es un buen ejemplo de lo que la sección 5 («Cómo se llama: la API») llama «las opciones son el nuevo *prompt*»: una pregunta ambigua hace fallar a todos.

15.3. ¿La respuesta se apoya en el documento?

Un texto del BOE, una pregunta sobre él y una respuesta; la pregunta al modelo es **si todo lo que afirma la respuesta está en el texto**. Serviría para vigilar a otros sistemas, por ejemplo un asistente que responde consultas con la documentación publicada. La mitad de las respuestas se apoyan en el texto y la otra mitad tiene un solo dato cambiado (una fecha, un importe, un código) o una afirmación añadida. Las escribió un modelo que no se mide en la prueba (Gemma 4 12B). 194 casos de prueba; 60 para entrenar las referencias.

Herramienta	Escalón	Nota	Acierto
Qwen3.5 9B, chat general con letras	5	9,4	97 %
Jeff-0.8B	4	8,4	91 %
Jeff-2B	4	7,8	88 %
Réplica de Jev de 2.000 millones	4	4,9	72 %
Kev-4B	4	4,9	77 %
Clasificador clásico	2	3,3	63 %
<i>Embeddings</i> y clasificador	3	2,5	58 %
Réplica de Jev de 800 millones	4	2,2	68 %
Kev-0.8B	4	1,0	61 %

Lo que enseña. Aquí importa para qué se entrenó cada modelo. Jeff se entrenó con ejemplos de esta tarea (respuestas que se apoyan o no en un documento) y su versión más pequeña, de 800 millones, saca un 8,4, muy por encima de Kev-4B, que tiene cinco veces más parámetros. El chat de 9.000 millones es el mejor. Las referencias apenas superan el azar: una fecha cambiada o una frase añadida no cambian casi nada las palabras.

Dos correcciones antes de dar las cifras por buenas. La primera versión de este caso la redactó Qwen3.5 9B, que también se mide: juzgaba respuestas escritas por él mismo. Se descartó entera y se rehízo con Gemma 4 12B, que no se mide. Después, al revisar a mano las respuestas, se encontraron tres en las que lo añadido para que la respuesta no se apoyara en el texto sí estaba en él; se excluyeron esos tres textos.

15.4. Cuánto tarda cada uno

Mediana de segundos por correo del triaje (tres preguntas cada uno), un modelo cada vez y con la máquina en reposo, en el Mac mini con M4 Pro y 24 GB:

Herramienta	Escalón	Segundos por correo
Clasificador clásico	2	menos de 0,01
<i>Embeddings</i> y clasificador	3	0,14
Kev-0.8B	4	0,19
Jeff-0.8B	4	0,36
Réplica de Jev de 800 millones	4	0,37
Jeff-2B	4	0,60
Kev-4B	4	1,10
Réplica de Jev de 2.000 millones	4	1,16
Qwen3.5 9B, chat general con letras	5	3,63

El clasificador clásico decide miles de veces más rápido que cualquier modelo. Entre los modelos de decisión, el tamaño manda: Kev-0.8B contesta las tres preguntas en unas dos décimas de segundo y Kev-4B, en algo más de un segundo. El chat de 9.000 millones tarda más de tres veces lo que Kev-4B, con una nota parecida. Cómo se midió, en la sección 6 de la metodología.

16. Fuera de la tarea: olvido y calibración

Ajustar un modelo con una tarea puede estropear lo que ya sabía hacer. Para comprobarlo, a los modelos ajustados con el BOE se les hicieron antes y después preguntas ajenas al BOE: 560 en inglés, de colecciones públicas (noticias, reseñas, consultas de clientes, inferencia) que no se usaron al ajustar, y 150 en español (tres colecciones públicas etiquetadas por personas). Se mira el acierto y, sobre todo, cuántas de las respuestas que da con un 90 % de seguridad o más resultan erróneas:

En las 560 preguntas en inglés	Kev-0.8B	Kev-4B
Acierto, antes y después del ajuste	83 % y 82 %	87 % y 86 %
Respuestas con ≥ 90 % que fallan, antes del ajuste	4 de 298 (1 %)	0 de 338 (0 %)
Después del ajuste	88 de 530 (17 %)	53 de 512 (10 %)
Después de recalibrar	30 de 425 (7 %)	11 de 399 (3 %)

El acierto no cambia más de lo que cambia por azar. Lo que cambia es la seguridad: el ajustado dice «90 %» a casi todo, también cuando se equivoca. Con un umbral como el de la sección 7 («Probabilidad, calibración y umbrales»), habría dejado pasar como seguras muchas respuestas erróneas. En español pasa lo mismo, y más acusado: con el 4B, fallan 7 de 58 respuestas seguras antes del ajuste, 35 de 132 después y 11 de 81 recalibrado.

La recalibración. Se hizo con 400 casos que no se usan para entrenar ni para medir: 140 textos del BOE de septiembre, 200 preguntas generales y 60 en español. La temperatura que mejor funcionó es 4, bastante más alta que la del modelo publicado:

- **En el 4B funciona.** En inglés, las respuestas seguras que fallan bajan del 10 % al 3 %, y en la prueba del BOE fallan 3 de 289. En español mejora mucho, pero sigue peor que antes del ajuste.
- **En el 0.8B no basta.** Necesitaría una temperatura de más de 7, y cada tipo de pregunta pide una distinta: unos 4 el BOE, 7 las preguntas generales y 18 el español. Una sola temperatura no puede corregir las tres a la vez. No se le deberían fijar umbrales.

La lección es general: **el ajuste no hace olvidar cómo responder, pero sí cuándo dudar**, y eso solo se ve midiendo la calibración fuera de la tarea ajustada. La primera versión de este experimento midió el olvido con 12 preguntas y concluyó que no había ningún problema; con 710 apareció. Y no es solo cosa del ajuste: en los casos de uso, casi todos los modelos sin ajustar pierden puntos por fallar con demasiada seguridad en alguna pregunta.

Límites de estas medidas. Un solo periodo de prueba en el BOE, conjuntos de prueba de unos cientos de casos y datos sintéticos en los casos de uso. Aún no se ha fijado ningún umbral: debería salir de otro conjunto aparte, distinto del de calibración y del de prueba. Las preguntas en inglés salen del conjunto de prueba con que se publica Kev, así que le favorecen frente a otros modelos.

17. Ajuste fino: enseñarle tus casos

Un modelo de decisión recién descargado sabe hacer juicios generales, pero no conoce los tipos de escrito, la terminología ni los criterios de tu organización. Con un modelo de lenguaje resolverías eso metiendo instrucciones y ejemplos en el *prompt*. Aquí no hay *prompt* libre donde meterlos: se le enseñan **entrenándolo un poco más** con casos propios. A todo entrenamiento que se hace sobre un modelo ya entrenado se le llama **post-entrenamiento** (*post-training*); la forma que está al alcance de una organización es el **ajuste fino** (*fine-tuning*).

17.1. Qué se entrena: LoRA

Un modelo como Kev-4B tiene unos 4.000 millones de números (los **pesos** o **parámetros**). Re-entrenarlos todos exige mucha memoria y mucho tiempo. La técnica habitual, **LoRA** (*low-rank adaptation*), congela todos esos pesos y entrena solo unas matrices pequeñas que se suman a algunas de las grandes:

$W_{\text{efectiva}} = W_{\text{congelada}} + B \cdot A$

A y B son estrechas: su «rango» (16 en Kev) fija su tamaño

Piensa en ello como un parche sobre un binario que no tocas: el modelo original queda intacto y lo que aprendes se guarda aparte. En la prueba de este texto, el parche ocupa 43 MB en el 0.8B y 130 MB en el 4B, en torno al 1 % de los parámetros. Junto al parche se entrena también la cabeza. Así, el ajuste cabe en una máquina propia.

17.2. Los datos: casos que ya existen

Cada decisión que una persona ha tomado y ha quedado registrada (el tipo asignado a un escrito, la unidad a la que se envió, si se pidió subsanación) es un ejemplo resuelto. No hay que etiquetar desde cero: hay que extraer y revisar. Cada caso es una línea de un fichero JSONL con el mismo formato que la petición a la API, más la respuesta correcta en `label`:

```
{
  "state": "... interpone recurso de alzada contra la resolución de ...",
  "questions": {
    "tipo": {
      "type": "choice",
      "instructions": "¿Qué tipo de escrito es?",
      "criteria": {
        "solicitud": null,
        "alegación": null,
        "recurso de alzada": null,
        "recurso de reposición": null,
        "consulta": null,
        "otro": null
      },
      "label": "recurso de alzada"
    }
  }
}
```

En `choice`, `label` es el nombre de la opción; en `null`, `true` o `false`; en `score`, la posición del nivel, empezando por 0.

17.3. El reparto: tres grupos que no se mezclan

Si pruebas tu código solo con los casos que tenías delante al escribirlo, pasará las pruebas aunque no funcione con casos nuevos. Con un modelo pasa lo mismo, y peor: puede **memorizar** los ejemplos. Por eso los casos se reparten en tres grupos:

Grupo	Parte orientativa	Para qué
Entrenamiento	70 %	Lo único que el modelo ve al aprender
Calibración	15 %	Para elegir la temperatura
Prueba	15 %	Para medir el resultado; no se mira hasta el final

Cuidado con las **fugas** (*data leakage*): si dos escritos del mismo expediente quedan uno en entrenamiento y otro en prueba, el modelo «reconoce» el caso y la medida sale mejor de lo que es. El reparto se hace por expediente, no por documento. Si los datos tienen fecha, lo más seguro es repartir por periodos: entrenar con lo antiguo y probar con lo reciente, que es lo que pasará en producción.

17.4. El entrenamiento y sus ajustes

Con Kev, entrenar es un comando (simplificado; el real añade los datos del modelo de partida):

```
python -m kev.train --init_from jaredpalmer/kev-4b \
  --data mis-casos/entrenamiento.jsonl \
  --suite evals/v7/decision-v7 --replay 160 \
  --epochs 3 --lr 2e-5 --batch 4 --lora 16 --out modelos/mi-ajuste
```

Qué significa cada ajuste:

- **--init_from**: se parte del modelo ya publicado, no del modelo base de Qwen, para conservar lo que ya sabe hacer.
- **--epochs** (épocas): cuántas pasadas completas se dan por los datos.
- **--lr** (*learning rate*, ritmo de aprendizaje): cuánto se mueven los pesos en cada paso. Si es demasiado alto, el modelo olvida lo que sabía; si es demasiado bajo, apenas aprende. Al partir de un modelo ya entrenado se usa uno pequeño.
- **--batch** (lote): cuántos ejemplos se procesan antes de mover los pesos.

- **--replay** (repaso): mezcla ejemplos del entrenamiento original con los tuyos. Un modelo ajustado solo con casos nuevos tiende a estropear lo que sabía (se llama **olvido catastrófico**); el repaso lo reduce, como mantener las pruebas de regresión mientras cambias el código.

Estos valores **se fijan antes de ver ningún resultado**. Si los vas cambiando hasta que la prueba sale bien, acabas ajustándolos a la prueba, igual que retocar un código hasta que pasa un test que falla de forma intermitente: el test pasa y el problema sigue ahí.

17.5. Después de entrenar: recalibrar y medir

Tras el ajuste, el modelo suele volverse demasiado seguro. Además, el entrenador de Kev guarda el modelo ajustado con temperatura 1, sin la corrección que traía el publicado. Hay que **recalibrarlo** con el grupo de calibración y después **medirlo** con el de prueba, comparando siempre con el modelo sin ajustar:

- **acierto en cada opción**, no solo el global: un modelo que dijera siempre «solicitud» podría acertar el 60 % y no servir para nada;
- **calibración**: si su seguridad se corresponde con su acierto;
- **olvido**: que siga respondiendo bien a preguntas ajenas a tu tarea que ya sabía responder, y que su seguridad siga siendo de fiar también ahí.

Si mejora de forma clara, se fija un umbral y se pasa a un piloto en el que el modelo propone y una persona confirma o corrige. Esas correcciones son, a su vez, nuevos ejemplos para el siguiente ajuste.

Cuántos casos hacen falta. La guía de Kev: de 100 a 200 sirven para probar el proceso, con resultados ruidosos; de 300 a 600, para un primer ajuste real; de 1.000 a 3.000, para algo que vaya a producción. Importa sobre todo tener bastantes **de cada opción**: si una opción es rara (un 3 % de los casos, pongamos), el modelo tenderá a ignorarla.

Un aviso útil: antes de ajustar, prueba el modelo **sin ajustar** con tu pregunta y tu grupo de prueba. A veces basta con escribir mejor las opciones, y el ajuste no compensa el trabajo.

17.6. Con pocos datos no funciona

En el BOE, con 200 ejemplos por opción, el ajuste funcionó. Hubo otra prueba con una pregunta más fina y muchos menos datos: qué hace un «conviene» en un texto. Puede hacer dos cosas muy distintas:

- **Dar un consejo práctico al lector** (se queda): «Antes de actualizar el sistema, conviene guardar una copia de los datos: si algo falla, se puede volver atrás». El lector sale sabiendo qué hacer y por qué.
- **Anunciar o subrayar lo que el texto va a decir** (se filtra): «Conviene señalar que la calibración no cambia la respuesta más probable». Si se quita «Conviene señalar que», la frase dice exactamente lo mismo: «La calibración no cambia la respuesta más probable».

El segundo uso es uno de los tics de los textos generados por modelos de lenguaje: una muletilla que no añade nada y que, repetida, delata el origen del texto. Un modelo que distinguiera los

dos usos serviría para detectar y filtrar esos excesos al revisar un texto, algo que no puede hacer una lista de palabras prohibidas, porque la palabra es la misma en los dos casos.

Los ejemplos salieron de revisiones reales de textos propios, que no se publican: 100, de los que solo 17 eran consejos prácticos. Con validación cruzada (cada caso se mide con un modelo que no lo vio al ajustarse), **el ajuste de Kev-0.8B no se distinguió del azar**: aprendió a responder casi siempre la opción mayoritaria. Un clasificador clásico con los mismos casos tampoco aprendió nada. Con 17 ejemplos de una opción no hay escalón que llegue: hacen falta **unos cientos de ejemplos de cada opción**.

18. Qué modelos existen

La idea la popularizó **Jev**, de la empresa TypeSafe AI (2026). Es un modelo cerrado: solo se puede usar como servicio en la nube, enviando los textos a sus servidores. Es muy barato por consulta, pero implica que los datos salen de la organización.

Poco después aparecieron **modelos abiertos** con el mismo planteamiento. Uno de ellos es **Kev**, con licencia Apache-2.0 y varios tamaños (de 0,8 a 27 mil millones de parámetros). Los pequeños funcionan en un servidor corriente o en un ordenador de sobremesa con buena memoria, sin conexión a internet. Eso permite:

- que **ningún dato salga** de la propia infraestructura;
- **no depender de un proveedor** ni de su precio;
- **ajustarlos con los propios casos**, con herramientas libres.

A finales de septiembre de 2026 apareció **Jeff**, un proyecto independiente con el mismo formato de petición que Jev, entrenado entero en máquinas propias (licencia MIT el código y Apache-2.0 los pesos), con versiones de 0,8 y 2 mil millones de parámetros. Sus autores avisan de que solo lo han pensado para inglés; en las pruebas de este informe, en español, funciona de forma desigual.

Los modelos probados en este informe, con su ficha técnica:

Modelo	Autor	Ficha técnica
Kev-0.8B y Kev-4B	Jared Palmer	huggingface.co/jaredpalmer/kev-4b
Jeff-0.8B y Jeff-2B	firelex	huggingface.co/mstrasser/Jeff-Qwen3.5-2B
Réplica de Jev, 2.000 millones	chaoliangUNSW	huggingface.co/chaoliangUNSW
Réplica de Jev, 800 millones	chaoliangUNSW	huggingface.co/chaoliangUNSW
Qwen3.5 9B (chat general)	Qwen (Alibaba)	ollama.com/library/qwen3.5

Son proyectos recientes y cambian rápido: conviene tratarlos como tecnología en evaluación, no como producto maduro.

19. Cómo empezar

1. **Pruébalo.** Clona el repositorio, arranca el laboratorio y un servidor de Kev (el README explica cómo) y hazle tus propias preguntas con el formulario o desde código.
2. **Elige una sola decisión** concreta y frecuente, que hoy se tome a mano, y escribe la pregunta y sus opciones exactamente como las usarías.
3. **Mide el modelo sin ajustar** con 100 o 200 casos ya resueltos por personas: acierto en cada opción y calibración. Mide también, con los mismos casos, un clasificador clásico: si lo iguala, no necesitas el modelo.
4. **Si no basta, ajústalo.** Reúne unos cientos de casos de cada opción, repártelos en entrenamiento, calibración y prueba sin fugas, fija los ajustes antes de mirar, entrena, recalibra y mide, también fuera de tu tarea.
5. **Fija un umbral** con casos apartados y haz un piloto en el que el modelo solo propone y una persona revisa.
6. **Decide con los datos del piloto** si merece la pena seguir.

PARTE VI · CIERRE

20. Veredicto provisional

El repositorio que acompaña a este texto es un **toolkit de evaluación de modelos de decisión**: un laboratorio para comparar modelos con los mismos casos y un conjunto de pruebas que se repetirá con cada modelo nuevo que salga. Esta valoración es **provisional**: refleja el estado de la tecnología a 29 de septiembre de 2026, con los modelos probados hasta ahora (Kev-0.8B y Kev-4B, sin ajustar y ajustados; Jeff-0.8B y Jeff-2B; las réplicas abiertas de Jev de 800 y 2.000 millones de parámetros; un modelo de chat general, Qwen3.5 9B; y dos referencias sin modelo de lenguaje, un clasificador clásico y *embeddings* con clasificador). El Jev original no se ha probado, porque exige enviar los textos a sus servidores.

En una frase: para preguntas que dependen del **significado**, los modelos de decisión abiertos de 2.000 a 4.000 millones de parámetros **ya son útiles con revisión humana**, en una máquina propia; para preguntas que se resuelven por el **vocabulario**, **un clasificador clásico basta** y es mucho más barato. Ninguno está para decidir solo.

Aspecto	Valoración	Por qué
Integración en una aplicación	Buena	Devuelven datos, no texto; no pueden salirse de las opciones; la API es sencilla y estable
Preguntas de vocabulario	No compensan	En el BOE, un clasificador clásico entrenado en segundos iguala a los modelos ajustados durante horas, y con 35 ejemplos supera a todos los modelos sin ajustar
Preguntas de significado, sin ajustar	Buena, pero desigual	78 % en PAWS-X sin haber visto la tarea; notas de 7,5 a 8,4 en correo y expedientes, donde las referencias sacan de 3 a 6,5. Cada modelo rinde en lo que se parece a su entrenamiento: Jeff-0.8B es flojo en correo y muy bueno comprobando respuestas (8,4)
Ajustados, con datos suficientes	Muy buena	95 % y 96 % en el BOE con 1.400 ejemplos; +20 puntos en PAWS-X con 200
Con pocos datos	No funciona	Con 17 ejemplos de la opción difícil, ni el ajuste ni el clasificador clásico se distinguieron del azar
Frente a un chat general	Empate en acierto, ventaja en coste	Un chat de 9.000 millones preguntado con letras rinde como Kev-4B, pero tarda de tres a cuatro veces más y ocupa más memoria
Fiabilidad de su seguridad	Frágil	Casi todos fallan demasiado con un 90 % o más; el ajuste lo empeora; recalibrando se recupera en el 4B, no en el 0.8B
Coste y soberanía	Muy favorable	Un ordenador de sobremesa, sin enviar nada fuera; respuesta en décimas de segundo o en un segundo
Español	Aceptable	Todas las pruebas son en español y funcionan, aunque la calibración empeora más que en inglés

Aspecto	Valoración	Por qué
Madurez	Baja	Proyectos de semanas (Jeff se publicó el 28 de septiembre de 2026); sus herramientas tienen límites que no avisan

Sobre la madurez, algunos ejemplos encontrados por el camino: el evaluador de Kev descartaba en silencio los textos largos (182 de 350 en la prueba del BOE), su herramienta de calibración tiene un tope que el 0.8B ajustado supera, y Jeff rechaza las preguntas de más de 26 opciones. Nada de eso impide usarlos, pero obliga a medir con cuidado y a no dar por buenas las cifras de nadie, incluidas las de este texto.

Qué elegir hoy:

- **Si la pregunta la resuelven las palabras y tienes ejemplos:** un clasificador clásico.
- **Si depende del significado y no tienes ejemplos:** Kev-4B, o un chat de 9.000 millones si el tiempo por consulta no importa.
- **Para comprobar si una respuesta se apoya en un documento:** Jeff-0.8B, pequeño y rápido, que se entrenó para eso, o un chat de 9.000 millones si el tiempo no importa.
- **Si tienes unos cientos de ejemplos de cada opción:** ajusta Kev-4B y recalíbralo. Kev-0.8B solo si la máquina no da para más, y sin fijarle umbrales.
- **En todos los casos:** mide contra el clasificador clásico antes de decidir, y no fijas umbrales sin recalibrar.

Qué falta para pasar de provisional a firme:

- probarlo con datos reales: un lote de expedientes con la respuesta conocida y la prueba privada con correos reales de un buzón general;
- fijar un umbral con casos apartados y medir cuánto trabajo ahorra y cuántos errores deja pasar;
- repetir las pruebas con cada modelo nuevo que salga, con los mismos datos, para ver si la tecnología madura.

21. Glosario

Modelo de decisión. Modelo que, dado un texto y una pregunta con opciones cerradas, devuelve una probabilidad por opción. No genera texto.

Clasificador clásico. Programa de aprendizaje automático anterior a los modelos de lenguaje que aprende qué palabras van con cada respuesta a partir de ejemplos etiquetados. En este informe, TF-IDF y regresión logística.

TF-IDF. Forma de convertir un texto en números: cuenta qué palabras aparecen y les da más peso cuanto más raras son en el conjunto de textos.

Regresión logística. Clasificador lineal que convierte una suma ponderada de rasgos en probabilidades.

Embedding. Vector de números que representa el significado aproximado de un texto, calculado por un modelo pequeño. Dos textos parecidos en significado dan vectores cercanos.

Caja de herramientas escalonada. La forma de elegir herramienta que propone este informe: reglas, clasificador clásico, *embeddings*, modelo de decisión y modelo de lenguaje, como los escalones de una caja que se abre en bandejas escalonadas; se sube al siguiente solo si el de abajo no llega.

Paráfrasis. Dos textos que dicen lo mismo con otras palabras.

Modelo de lenguaje (LLM). Modelo que genera texto *token a token*: los chats como ChatGPT o Claude.

Token. Trozo de palabra con el que trabajan los modelos de lenguaje.

Prompt. Texto libre con instrucciones y contenido que se le escribe a un chat. Los modelos de decisión no lo usan: reciben campos fijos (texto, pregunta, opciones).

Cabeza. Pieza pequeña que se añade a un modelo de lenguaje para que, en vez de escribir, puntúe cada opción.

Logit. Puntuación sin normalizar que el modelo da a cada opción. Softmax los convierte en probabilidades.

Softmax. Función que convierte una lista de números en probabilidades que suman 1: eleva e a cada número y divide por la suma.

Log-loss. Menos el logaritmo de la probabilidad dada a la respuesta correcta, en media. Cuanto más baja, mejor; castiga mucho equivocarse con seguridad.

Calibración. Grado en que la probabilidad que da el modelo coincide con su acierto real. Bien calibrado: cuando dice 80 %, acierta 8 de cada 10.

ECE. Error de calibración esperado: diferencia media entre la seguridad y el acierto, por tramos de seguridad.

Temperatura. Número por el que se dividen los *logits* antes de softmax. En un modelo de decisión corrige lo seguro que se muestra sin cambiar la respuesta; en un modelo de lenguaje controla la aleatoriedad al generar.

Umbral. Probabilidad a partir de la cual se acepta la respuesta del modelo sin revisión.

Post-entrenamiento. Cualquier entrenamiento que se hace sobre un modelo ya entrenado. El ajuste fino es un caso.

Ajuste fino. Seguir entrenando un modelo ya hecho con ejemplos propios, para que responda mejor en una tarea concreta.

LoRA. Técnica de ajuste fino que congela el modelo y entrena solo unas matrices pequeñas que se suman a las grandes. Reduce mucho la memoria y el tiempo necesarios.

Época. Una pasada completa por los datos de entrenamiento.

Ritmo de aprendizaje (*learning rate*). Cuánto se mueven los pesos en cada paso del entrenamiento.

Repaso (*replay*). Mezclar ejemplos del entrenamiento original con los nuevos para que el modelo no olvide lo que sabía.

Olvido catastrófico. Cuando un modelo, al aprender algo nuevo, empeora en lo que ya sabía hacer.

Fuga de datos (*data leakage*). Cuando información de la prueba se cuela en el entrenamiento y la medida sale mejor de lo que es.

Bootstrap. Forma de estimar cuánto se debe al azar una cifra: se repite el cálculo sobre muchos sorteos con repetición de los mismos casos.

Intervalo de confianza del 95 %. Rango en el que cae la cifra en el 95 % de esos sorteos. Si una diferencia entre dos modelos no incluye el cero, no se explica por azar.

Pesos abiertos. Modelo cuyos ficheros se publican y se pueden descargar y ejecutar en máquinas propias.

GB y GiB. Dos unidades de memoria que se confunden a menudo. Un gigabyte (GB) son mil millones de bytes (10^9); un gibibyte (GiB), 1.073.741.824 bytes (2^{30}), un 7,4 % más. Los fabricantes de discos y macOS usan GB; Windows y muchos programas muestran GiB aunque escriban «GB». Las memorias de este informe están en GB decimales, salvo los «24 GB» del equipo, que son la cifra comercial de su memoria y equivalen a 24 GiB. La norma que distingue ambas unidades es la ISO/IEC 80000-13.

Créditos

Modelos de decisión y ajuste fino

Una introducción para programadores que ya han usado modelos de lenguaje

Joaquín Herrero Pintado

Creative Codeworks

Versión 2.1 · 29 de septiembre de 2026

Primera versión: 28 de septiembre de 2026 · Cambios de cada versión: [CAMBIOS.md](#)

Última versión: creativecodeworks.com/toolkit-modelos-de-decision.html

Toolkit de evaluación de modelos de decisión: github.com/joakinen/toolkit-modelos-de-decision
creativecodeworks.com

© 2026 Joaquín Herrero Pintado.

Este texto se publica con licencia Creative Commons Reconocimiento-CompartirIgual 4.0 Internacional (CC BY-SA 4.0): puedes copiarlo y adaptarlo, también con fines comerciales, siempre que indiques la fuente (autor, título y creativecodeworks.com) y publiques lo que hagas a partir de él con la misma licencia. El código del toolkit tiene licencia Apache-2.0.

Los textos de la prueba del BOE y del caso de respuestas apoyadas proceden de la Agencia Estatal Boletín Oficial del Estado (boe.es), reutilizados según sus condiciones de datos abiertos; los pares de paráfrasis, de PAWS-X (Google Research); los controles en español, de XNLI, PAWS-X y reseñas de Amazon. Los correos y expedientes de los casos de uso son sintéticos, escritos para esta prueba. Los modelos probados son de sus autores: Kev, de Jared Palmer; Jeff, de firelex; las réplicas abiertas de Jev, de chaoliangUNSW; Qwen3.5, de Alibaba; el modelo de *embeddings* bge-m3, de BAAI; y Gemma 4, de Google, que escribió las respuestas del caso de respuestas apoyadas. Jev es un modelo de TypeSafe AI, que no está relacionada con este trabajo.

Este informe y el toolkit que lo acompaña se han hecho con la asistencia de **Claude Code** (Anthropic). La herramienta se usó para escribir y ejecutar los programas de evaluación, lanzar los ajustes y las medidas, contrastar cada cifra con los datos de los que sale, revisar a mano muestras de resultados, redactar borradores del texto y maquetarlos. Las preguntas, el criterio sobre qué medir y qué dar por bueno, y la decisión de publicar también lo que corrige versiones anteriores son míos; míos son también los errores.

Este informe se actualizará cada vez que el toolkit evalúe un modelo de decisión nuevo.